SOC / ENTERPRISE PENTEST HOMELAB

SOC Homelab
Master Documentation

Samengevoegde, gededupliceerde referentie voor het volledige SOC Homelab project — architectuur, systemen, beslissingen, guides, troubleshooting-historie en tijdlijn.

Auteur: Joost Hebly · **Samengesteld:** 13 juli 2026 · **Bron:** git repo /Homelab (docs/)

Betrouwbaarheidslegenda: ✅ geverifieerd deze sessie / harde evidentie · ⚠️ niet opnieuw geverifieerd, overgenomen uit eerder document · ❌ gepland / nog niet gebouwd.

Table of contents

1. Project overview & purpose

SOC Homelab (originally named **"Fortress Bazzite"**) is Joost Hebly's private cybersecurity training environment. It exists to learn how a real Security Operations Center works by building and operating one — including the real-world problems that come with it (firewalls too strict, clocks that won't sync, agents that won't check in).

Focus areas:

- **Blue Team**: security monitoring, SIEM use (Security Onion/Kibana), incident response, log analysis, network visibility (traffic mirroring), detection engineering.
- **Red Team**: vulnerability testing, exploitation practice, attack simulation, Active Directory security testing.
- Realistic **Active Directory** identity environment and general **network defense**.

The project doubles as a portfolio piece — Joost produces formal deliverables from it (project reports, daily reports, technical documentation, a portfolio document) alongside the working lab itself.

Origin: from "Fortress Bazzite" to this lab

The project began as **Fortress Bazzite**, captured in an early design document (`Fortress_Bazzite_joost-hebly_rapport_network_security_IDS_IPS.docx`, 2026-07-05). That plan described a self-built combination of Suricata (NIDS), Zeek (network telemetry), Wazuh (host monitoring), and a custom dashboard (system health, security and gaming metrics).

In practice, **Security Onion** was chosen as the platform instead. This isn't a change of goal, just a different way to reach it: Security Onion already bundles Suricata and Zeek, and uses Elastic Agent/Fleet for host monitoring — the role Wazuh would have filled. The original detection goals from the Fortress Bazzite document (port scans, brute force, reverse shells, web attacks, etc.) remain the target; see §6.

AI collaboration model

Two assistants are used deliberately for different roles:

- **ChatGPT** — architecture, planning, troubleshooting strategy.
- **Claude Code** — terminal operations, documentation, scripts, implementation.

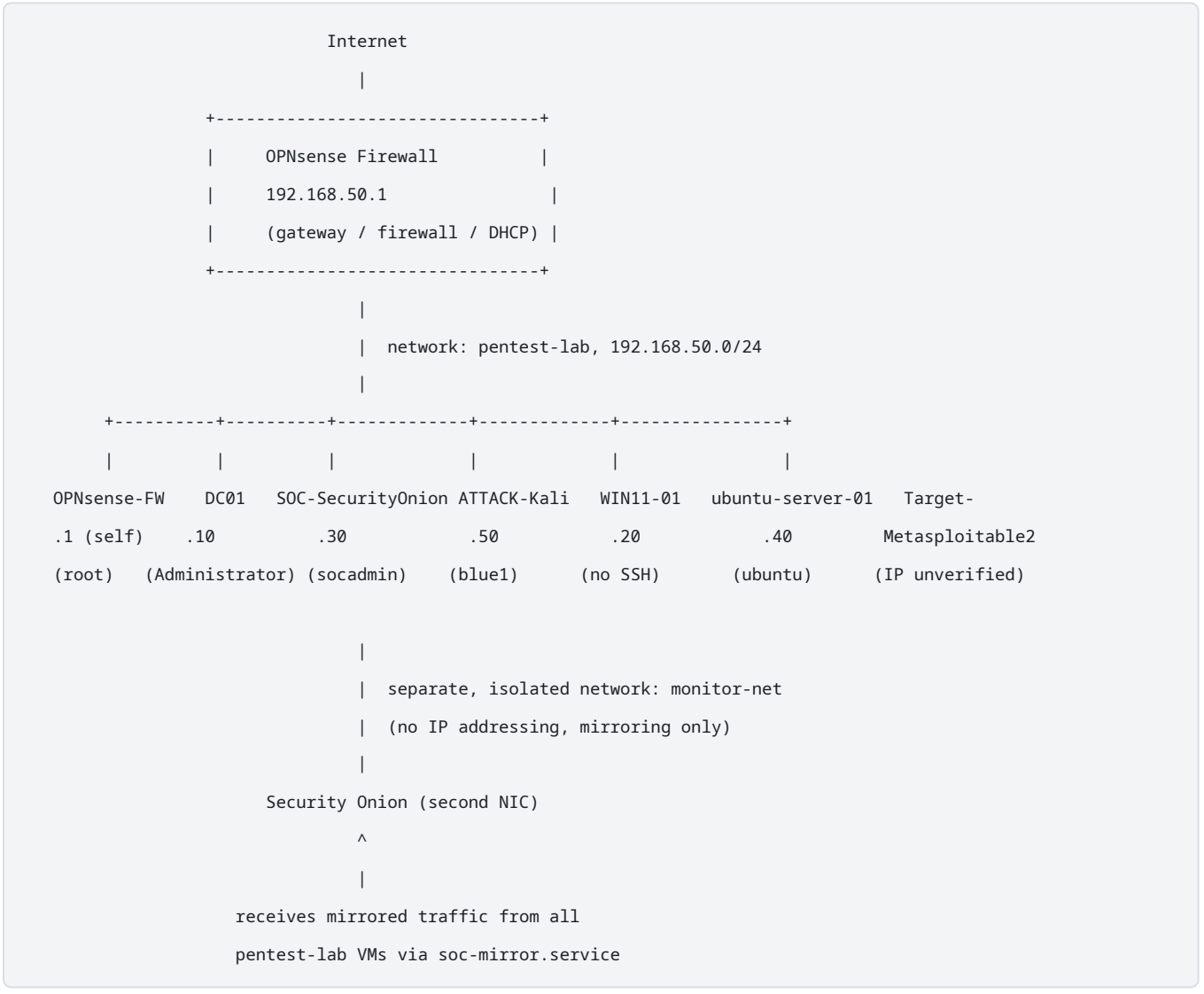
AI assistants operate under a written access policy (see §4): read documentation first, never store secrets, explain before changing infrastructure, ask before destructive actions, document every important change.

Golden rule (from the top-level README): No destructive changes without approval. All changes must be documented.

---

2. Architecture & network design

Network map



Security Onion has **two network interfaces**: one on `pentest-lab` (192.168.50.30, for Fleet/Kibana/SSH/web UI) and one on `monitor-net` (no IP addressing, receives only mirrored traffic — see "Traffic mirroring" below).

IP address table (verified 2026-07-13)

IP	System	virsh VM name	SSH alias	SSH user	Status
192.168.50.1	OPNsense (firewall/gateway)	`OPNsense-FW`	`opnsense`	`root`	✅
192.168.50.10	DC01 (Active Directory, PDC Emulator, domain `pentest.lab`)	`DC01`	`dc01`	`Administrator`	✅
192.168.50.20	WIN11-01 (Windows 11 workstation)	`WIN11-01`	*(none — no SSH server)*	—	✅ (IP), ⚠️ (exact role undocumented)
192.168.50.30	Security Onion 3.1.0 standalone (SIEM/IDS/Fleet)	`SOC-SecurityOnion`	`security-onion`	`socadmin`	✅
192.168.50.40	ubuntu-server-01 (general Linux server)	`ubuntu-server-01`	`ubuntu-server`	`ubuntu`	✅ (IP), ⚠️ (exact role undocumented)
192.168.50.50	Kali Linux (Red Team workstation)	`ATTACK-Kali` ⚠️ *leading space in the name — see below*	`kali`	`blue1`	✅
unverified	Metasploitable2 (intentionally vulnerable target)	`Target-Metasploitable2`	*(none)*	—	⚠️

⚠️ **Known naming bug:** the VM name `ATTACK-Kali` begins with a literal space character in libvirt. This previously caused a real bug in scripts that matched VMs by name (see `soc-mirror.sh` history in §7). New scripts should either account for the space or, better, match on UUID instead of name.

⚠️ **Conflicting historical IPs (resolved):** Several earlier documents (the English-language `SERVERS.md` , `ACTIVE_DIRECTORY.md` /chat-history archives, and the original `virtualization.md` / `opnsense_setup.md` / `security_onion_setup.md` guides) list **Security Onion at 192.168.50.20** — this was corrected on 2026-07-13; `.20` is actually WIN11-01, and Security Onion is `.30` . Similarly, the 2026-07-09 daily report's DHCP plan assigned Kali to `.20` and Metasploitable2 to `.50` ; the verified 2026-07-13 reality is Kali on `.50` and WIN11-01 (not in the original plan) on `.20` . Treat `docs/ASSET_INVENTORY.md` / `NETWORK.md` (both 2026-07-13) as authoritative over any older document for IPs.

Virtual networks (libvirt, verified via `virsh net-list --all`)

Network	Status	Purpose
`pentest-lab`	active	Main network, 192.168.50.0/24, isolated. No libvirt-native DHCP — OPNsense handles this exclusively (see §7 for why).
`monitor-net`	active	Isolated, no IP addressing, exclusively for traffic mirroring to Security Onion.
`default`	active	Standard libvirt network, not used by lab VMs.

Physical host

Bazzite Linux — the physical machine running everything.

⚠️ The following hardware specs come from the original Fortress Bazzite design document (2026-07-05) and were not reverified this session:

- CPU: Intel Core i9-11900K
- GPU: NVIDIA RTX 3090
- WiFi: Intel AX210 (Wi-Fi 6E), PCI-passthrough to Kali for monitor-mode testing
- RAM: 64GB (per `docs/guides/host_setup.md`)
- Virtualization: KVM/QEMU/libvirt/virt-manager
- Also runs Docker (for OWASP Juice Shop and other containers)

Traffic mirroring (`soc-mirror.service`)

Security Onion sees network traffic from the other lab VMs via an event-driven mirroring mechanism (`scripts/soc-mirror.sh`, triggered by a libvirt qemu hook on every VM start/stop):

1. When a VM on `pentest-lab` starts, its traffic is mirrored to Security Onion's `monitor-net` interface.
2. When Security Onion itself restarts (getting a new network interface), stale mirror rules are automatically cleaned up and recreated.
3. No periodic timer — everything is driven by VM start/stop events. Check status with:

```
scripts/soc-mirror.sh --status
```

A real libvirtd deadlock bug was found and fixed in the hook that drives this — see §7.

DNS and DHCP

- **DHCP:** handled entirely by OPNsense (libvirt's own DHCP for `pentest-lab` was deliberately removed — see §7).
- **DNS:** DC01 (Active Directory DNS) for the `pentest.lab` domain; OPNsense forwards external queries.
- ⚠️ Exact DHCP ranges and DNS forwarders are not documented in detail — a possible follow-up.

Security principles baked into the network

- **Segmentation** — `monitor-net` is fully separated from `pentest-lab`; only Security Onion sits on both.
- **Least privilege** — Security Onion's firewall hostgroups ensure each system can only reach the ports it actually needs.
- **Monitor first** — from the original Fortress Bazzite design: full visibility before additional blocking or automated response.
- **Documentation required** — every infrastructure change is recorded (`docs/daily/` , `docs/troubleshooting/`).

Planned network improvements (from the original Fortress Bazzite plan, not yet built)

VLAN segmentation · a separate management network · a separate attack network · additional Windows clients · honeypots · more SOC sensors.

---

3. Systems in detail

3.1 Bazzite Linux host & virtualization

Role: Physical virtualization host running all isolated lab VMs, while also serving as Joost's daily desktop/gaming machine.

Why Bazzite: modern Linux desktop, good hardware support, NVIDIA GPU support, gaming capability, and full access to native Linux virtualization tooling — the same machine covers daily computing, gaming, and lab hosting. Lesson learned: a desktop Linux system can successfully function as a serious home virtualization host when properly configured.

Stack: KVM (kernel virtualization) + QEMU (hardware emulation) + libvirt (management layer) + virt-manager (GUI). Chosen over VMware Workstation / VirtualBox / Proxmox for native Linux integration and to understand the technology directly, at some ergonomics cost.

Storage: VM disks use qcow2 (snapshot support, thin provisioning). One VM image was converted from VMDK: `qemu-img convert -f vmdk -O qcow2 source.vmdk destination.qcow2`.

Networking note: an early VM (`Pentest-Kali`, actually Debian 12, not real Kali) hit VirtIO incompatibility (initramfs hangup) with an old Metasploitable2 kernel — fixed by switching that VM to i440FX chipset + SATA disk + e1000 NIC. See §8 for full history.

Basic VM lifecycle:

```
virsh list --all
virsh start VMNAME
virsh shutdown VMNAME       # graceful; use --destroy only if truly stuck
virsh domdisplay VMNAME     # SPICE display connection
```

QEMU guest agent: not configured on VMs by default — `virsh qemu-agent-command` fails with `QEMU guest agent is not configured` until the in-guest agent is installed, enabled, and the channel is configured (see §7).

3.2 OPNsense (firewall/gateway)

Role: Central firewall and network gateway — the security boundary between the outside and the internal 192.168.50.0/24 lab network. Provides firewall, routing, DHCP, DNS forwarding, and network segmentation.

Why OPNsense: open-source, enterprise-style feature set, realistic firewall/routing/DHCP/DNS practice, good learning value for firewall troubleshooting and policy management.

IP: 192.168.50.1 (LAN interface). SSH alias `opnsense`, user `root` (password-only — no key auth set up as of the last check).

History note: first installed 2026-07-07; the first install attempt hung in "live media mode" (never actually installed to the VirtIO disk) and was redone with ZFS. WAN firmware-update/internet

connectivity had a DHCP/gateway problem that took until 2026-07-09 to fully resolve, alongside a separate libvirt-vs-OPNsense DHCP conflict (see §8).

3.3 DC01 — Active Directory Domain Controller

Role: Primary (and only) Windows Server Domain Controller.

Verified facts (2026-07-13):

- OS: Windows Server 2022 Standard Evaluation
- Domain: `pentest.lab` — DC01 is the domain's **PDC Emulator** (its only DC)
- IP: 192.168.50.10 (static)
- SSH alias `dc01`, user `Administrator` (username best-guess, not fully verified; port confirmed open)
- Timezone: `W. Europe Standard Time` (Amsterdam/CEST), set 2026-07-13 for local-time readability in Event Viewer/file timestamps — the underlying UTC clock remains authoritative for correlation with Security Onion

⚠️ **Documentation gap:** `ACTIVE_DIRECTORY.md` (the older root-level doc) still lists the domain name as "TO BE DOCUMENTED" and describes the OU/security-group structure as only "planned." In reality the domain (`pentest.lab`) has existed and been operational since 2026-07-10; the detailed OU/GPO structure below is still aspirational, not yet built.

Services: AD Domain Services, DNS, authentication, user/group management, Group Policy (planned in detail, not yet built out).

Planned domain structure (not yet implemented): OUs for Users / Administrators / Servers / Workstations / Security Groups. Planned account tiers: Domain Administrator, Server Administrator, SOC Administrator, Standard Users — following least-privilege separation.

Monitoring integration (built 2026-07-13): Elastic Agent (build `9.3.3+build202604082258`) enrolled in Security Onion's Fleet; Sysmon `15.21` (schema 4.91) installed with the SwiftOnSecurity config (schema 4.50, still compatible). Full story and root causes in §7.

Time sync: uses `pool.ntp.org`. The Windows `vmictimesync` hypervisor-integration service was found fighting with NTP at every boot and is now disabled, with a scheduled task forcing an NTP resync 30s after startup (see §7).

3.4 WIN11-01 — Windows 11 workstation

IP: 192.168.50.20. No SSH server installed (normal for a Windows 11 client) — not reachable via an SSH alias. ⚠️ Its precise role in the lab is not yet documented in detail — a possible follow-up.

3.5 ubuntu-server-01 — general Linux server

IP: 192.168.50.40. SSH alias `ubuntu-server`, user `ubuntu` (username best-guess). ⚠️ Precise role not yet documented in detail. Previously hosted Docker + OWASP Juice Shop during the earlier Fortress Bazzite phase.

3.6 Security Onion — SOC platform

Role: Central Security Operations Center monitoring platform — SIEM, IDS, Network Security Monitoring, log collection, alert investigation, threat detection.

Verified facts (2026-07-13):

- Version: Security Onion 3.1.0, standalone installation
- VM name (virsh): `SOC-SecurityOnion`
- IP: 192.168.50.30 (⚠️ several older docs incorrectly list `.20` — see the conflict note in §2)
- SSH alias `security-onion`, user `socadmin` (key auth already working)
- OS-level timezone intentionally left on UTC (`Etc/UTC`) — standard practice since Elasticsearch is UTC-internal regardless; the web UI (Hunt, Kibana) already renders browser-local time (confirmed `+02:00`/CEST). Aligning the OS-level timezone to `Europe/Amsterdam` for raw SSH-log readability is a low-priority open item, blocked on root access beyond the narrow `so-firewall`-scoped sudo currently granted.

Own firewall (separate from OPNsense): Security Onion has its own internal firewall gating which IPs can reach which of its own ports, based on **hostgroups** (named IP groups) mapped to **portgroups** (named port groups). Membership in one hostgroup does **not** grant access to another hostgroup's ports — this was the root cause of the entire DC01 Fleet outage (see §7). Full reference: §5.4.

Bundled capabilities: Suricata (network intrusion detection), Zeek (network telemetry/connection logs), Elasticsearch (log storage, UTC-internal), Kibana, Fleet (agent management), Hunt (Security Onion's own search UI, similar to Kibana Discover).

Components delivered by Elastic Agent on endpoints: Beats-equivalent log shipping, Windows Event Log / Sysmon collection, Elastic Defend (endpoint security/EDR, uses its own connection on port 3765 via the `endgame` hostgroup, separate from ordinary log ingest).

3.7 Kali Linux — Red Team workstation

Role: Penetration testing workstation. VM name `ATTACK-Kali` (leading space — known bug, see §2). IP 192.168.50.50. SSH alias `kali`, user `blue1` (key auth working). Used for vulnerability scanning, exploitation testing, network analysis — authorized testing within this isolated lab only.

Early in the project this VM was actually Debian 12 (named `Pentest-Kali`) rather than a genuine Kali install; a real Kali deployment is implied by the current asset inventory naming, though the exact re-install point isn't separately documented.

3.8 Metasploitable2 — vulnerable training target

VM name `Target-Metasploitable2`. ⚠️ Current IP not verified this session (the 2026-07-09 DHCP plan assigned it `.50`, which Kali now occupies — the assignment likely changed since). Intentionally vulnerable, used for exploitation/detection-testing practice. Does not respond to ACPI shutdown (no `acpid` on that old image) — `lab-stop.sh` always ends up force-stopping it via `virsh destroy` after a 60s timeout; this is expected, not a bug.

---

4. Architecture & security decisions

Why these technologies were chosen

Decision	Choice	Core reason
Host OS	Bazzite Linux	Already provided modern Linux + NVIDIA support + gaming capability + virtualization tooling on one machine
Virtualization	KVM/QEMU/libvirt/virt-manager	Native Linux virtualization, full control, deeper technical understanding than a turnkey hypervisor
Firewall	OPNsense	Realistic enterprise firewall/routing/DHCP/DNS practice
SOC platform	Security Onion	Complete, realistic SOC platform (SIEM+IDS+NSM) in one bundle, superseding a hand-rolled Suricata/Zeek/Wazuh stack
Identity	Windows Server AD (DC01)	Most enterprise environments run AD — needed for realistic detection/attack scenarios (failed logins, privilege escalation, lateral movement)
Version control	Git	Version history, recovery points, professional change-tracking workflow
Secrets storage	Separate `Secure/` container (`SOC-Secure.img`), never in Git	Documentation can be shared; secrets must never enter Git history (removing a secret later doesn't remove it from history)
AI assistance	ChatGPT (architecture/planning) + Claude Code (implementation/docs)	Support tools, not decision-makers — bounded by the AI Access Policy below

AI Access Policy (binding on any AI assistant working on this project)

Source of truth for AI behavior: `README.md` , `PROJECT_RULES.md` , `AI_ACCESS_POLICY.md` , `docs/INDEX.md` , `docs/guides/` , `docs/troubleshooting/` , `docs/decisions/` .

Allowed: read documentation, analyze architecture/configurations, explain commands, suggest improvements, help write documentation, help troubleshoot.

Restricted — AI must never:

- Store or remember passwords, secrets, API keys, private keys, recovery codes, tokens
- Commit credentials to Git
- Modify firewall rules or critical infrastructure without approval
- Delete systems/files or make destructive changes without confirmation

Change procedure (before any infrastructure change): 1) explain what will change, 2) explain possible risk/impact, 3) create a backup or snapshot, 4) execute, 5) document the result.

Credential handling: credentials are never placed in Git or normal markdown files. The user enters passwords manually when required. Sensitive data lives in `Secure/SOC-Secure.img` , opened manually by the administrator only.

Troubleshooting method (mandated order): check existing documentation → identify the affected system → review previous troubleshooting cases → explain the cause → apply the smallest safe change → document the result.

Core security principles (from `docs/decisions/security_decisions.md` and `SECURITY.md`)

- **Least privilege** — separate admin vs. daily accounts; systems get only the permissions they need.
- **Defense in depth** — multiple security layers (OPNsense's boundary firewall + Security Onion's own hostgroup firewall + host-level controls).
- **Network isolation** — the whole 192.168.50.0/24 lab is isolated specifically because it contains attack tooling and deliberately vulnerable machines.
- **Documentation first, automation second** — every important change is recorded before/alongside being automated.
- **Backups before changes** — snapshots required before major infrastructure modifications.
- **Monitoring is core, not an add-on** — Security Onion provides the visibility foundation; "you can't improve what you can't see."
- **Controlled testing only** — all offensive testing stays inside the isolated lab boundary.

Incident response phases (from `SECURITY.md`, operationalized in the runbook — §6.2)

1. **Identification** — what happened, which systems, severity.
2. **Analysis** — collect logs, alerts, system info.
3. **Containment** — isolate affected systems, block malicious activity.
4. **Recovery** — restore services, verify security.
5. **Lessons learned** — document root cause, improvements, prevention.

---

5. Operational guides

5.1 Starting/stopping the lab

Preferred — desktop launchers (see §5.5):

- **Pentest Lab Start** — starts all 7 VMs in the correct order.
- **Pentest Lab Stop** — stops all VMs cleanly.

Command line:

```
~/Homelab/scripts/lab-start.sh
~/Homelab/scripts/lab-stop.sh
```

Single VM:

```
virsh -c qemu:///system start <vm-name>
virsh -c qemu:///system shutdown <vm-name>   # use shutdown, not destroy, unless truly stuck
virsh -c qemu:///system list --all
```

Watch for the exact VM names in §9 — note the leading space in `ATTACK-Kali`.

`lab-start.sh` starts all 7 VMs in order (OPNsense-FW, DC01, SOC-SecurityOnion, `ATTACK-Kali`, WIN11-01, ubuntu-server-01, Target-Metasploitable2), is idempotent (already-running VMs skipped), and wraps every `virsh` call in `timeout` so a stuck daemon can't hang the script.

`lab-stop.sh` stops clients/targets first, OPNsense and Security Onion last; polls each VM via ACPI shutdown, only force-killing via `virsh destroy` after a per-VM timeout (60s normal, 180s for Windows/Security Onion) and a clear warning; prints a final graceful-vs-forced summary.

5.2 Connecting to systems

Via the **SSH Alle Machines** launcher (opens one Konsole window with a real, independent tab per machine — see §5.5), or directly:

```
ssh opnsense
ssh dc01
ssh security-onion
ssh kali
ssh ubuntu-server
```

For GUI VM management: the **Homelab VM Manager** launcher opens virt-manager connected to `qemu:///system` (not the unrelated `qemu:///session` connection).

SSH key status: only `kali` and `security-onion` currently accept the local key passwordlessly. `opnsense`, `dc01`, and `ubuntu-server` prompt for a password (confirmed working interactively) — run `ssh-copy-id <alias>` on any of them to enable passwordless login.

5.3 Security Onion / Kibana / Fleet in the browser

Via the **Security Onion Operator** desktop launcher, which opens a persistent, dedicated Chromium profile with every important page as its own tab (Overview, Hunt, Detections, Cases, Grid, Administration, PCAP, Kibana, Fleet). Built at `~/Homelab/browser/` (Playwright-based); see full design rationale below.

Why a persistent "daemon" browser instead of a fresh one per command: one long-running browser process exposes a local Chrome DevTools Protocol port (`127.0.0.1:9223` , never exposed beyond localhost); every tool (operator, audit script, future modules) attaches to that same process via CDP instead of launching its own. Only the process that started the daemon may close it.

First login: `scripts/soc-browser.sh` starts the daemon headed and waits (up to 10 minutes) for the login screen to disappear — it never sees or handles the password. Security Onion's own portal and Kibana/Fleet ("Welcome to Elastic") are **two separate logins**; expect to log into both once. After that, the session persists via the browser's own on-disk profile (`browser/profile/` , never in Git) for as long as the daemon runs or the profile exists — no script in this repo reads or copies cookies.

Revoking access: log out inside the browser (ends the session, profile files remain), or delete `browser/profile/` entirely (full reset).

⚠️ **A security incident during development (recorded, not swept under the rug):** a script once used Playwright's `context.request` API to call a Fleet endpoint; the failed call's error/log formatting printed the full request headers — **including the live session cookie** — into terminal output visible in that session's transcript. Not a leak to any third party (private, isolated-network conversation), but exactly the kind of exposure this project exists to avoid. Response: the affected daemon was restarted immediately (invalidating that session), and every API call now goes through `lib/browser.mjs` 's `fetchJsonInPage()` , which runs `fetch()` *inside* the already-authenticated page via `page.evaluate()` — cookies are attached by the browser itself and never pass through this project's Node code or its error formatting. `context.request` (or any raw-HTTP-call API outside the page) should not be reintroduced without the same care.

Read-only guarantee: everything built so far (navigation, screenshots, the audit's GET-only API calls) is read-only. Nothing can modify Fleet, agent policies, detection rules, ingest pipelines, Elasticsearch, Kibana saved objects, or Security Onion's own configuration — any future module that *would* touch those needs a proposal and explicit approval first.

Fleet status without touching the browser manually:

```
~/Homelab/scripts/soc-web-audit.sh
```

Generates a Markdown report in `browser/artifacts/` covering: SO/Kibana reachability and login state, per-agent Fleet status (online/offline/unhealthy, DC01 called out specifically), data-stream freshness (Windows Event Logs, Sysmon, Suricata), Grid member status, and a Detections-page summary. Requires the daemon already running and logged in. Known gaps: Cases/Hunt event-level content isn't searched yet (Fleet/data-stream metadata only); SSH correlation with actual OS-level state isn't automated; ingest-pipeline detail isn't pulled (Kibana's `index_management` API returned "not available" on this install).

5.4 Network ports & firewall hostgroups reference

Security Onion's **own** firewall (separate from OPNsense) — reconstructed 2026-07-13 directly from `/opt/so/saltstack/default/salt/firewall/defaults.yaml` and `soc_firewall.yaml`, because this exact gap caused the DC01 outage (see §7.6).

Portgroups → ports:

Portgroup	Port(s)	Purpose
`elastic_agent_control`	TCP 8220	Fleet Server checkin
`elastic_agent_data`	TCP 5055	Data ingest (logs/events)
`elastic_agent_update`	TCP 8443	Agent updates
`endgame`	TCP 3765	Elastic Defend/Endpoint output ("Endgame" = Elastic's historical product name)
`beats_5044` / `beats_5644` / `beats_5066` / `beats_5056`	TCP 5044 / 5644 (SSL) / 5066 / 5056	Logstash beats input
`kibana`	TCP 5601	Kibana web UI
`elasticsearch_rest`	TCP 9200	Elasticsearch REST API
`elasticsearch_node`	TCP 9300	Elasticsearch node-to-node
`docker_registry`	TCP 5000	Internal Docker registry
`influxdb`	TCP 8086	Metrics
`postgres`	TCP 5432	PostgreSQL
`all`	TCP/UDP 0–65535	Fully trusted hostgroups (e.g. `anywhere`)

Key hostgroups:

Hostgroup	Typically for	Grants access to
`analyst`	People using the web UI	nginx web UI (443), Kibana (5601)
`fleet`	Fleet Server itself	Various Fleet-related ports
`elastic_agent_endpoint`	Windows/Linux machines running Elastic Agent	Fleet checkin (8220)
`beats_endpoint` / `beats_endpoint_ssl`	Machines shipping logs via beats	Data ingest (5055 + beats ports), TLS variant
`endgame`	Machines with Elastic Defend	Endpoint output (3765)
`elasticsearch_rest`	Systems talking directly to Elasticsearch	9200
`manager` / `receiver` / `sensor` / `standalone`	Internal Security Onion grid roles	Role-specific ports
`desktop`	Admin workstations	Like `analyst`, plus extras
`customhostgroup0` – `9`	Free to define	—

Rule of thumb — what a new endpoint needs:

Want to...	Need hostgroup
View the web UI/Kibana	`analyst`
Have Elastic Agent check into Fleet	`elastic_agent_endpoint` (or `fleet`)
Ship logs via Elastic Agent	`beats_endpoint`
Get Elastic Defend telemetry working	`endgame`

A normal Windows/Linux endpoint with Elastic Agent needs *at minimum* all three of `elastic_agent_endpoint`, `beats_endpoint`, and `endgame` — `analyst` alone is not enough, even though that seems logical. Membership in one hostgroup grants nothing in another.

Adding a system to a hostgroup (on Security Onion, as root):

```
sudo so-firewall includehost <hostgroup-name> <ip-address>
sudo so-firewall apply
```

Example (what was done for DC01 on 2026-07-13):

```
sudo so-firewall includehost elastic_agent_endpoint 192.168.50.10
sudo so-firewall includehost beats_endpoint 192.168.50.10
sudo so-firewall includehost endgame 192.168.50.10
sudo so-firewall apply
```

This is **additive** — it does not remove the IP from hostgroups it's already in. To remove entirely: `sudo so-firewall removehost <ip>`.

Read-only inspection (no root needed):

```
cat /opt/so/saltstack/default/salt/firewall/defaults.yaml
cat /opt/so/saltstack/default/salt/firewall/soc_firewall.yaml
cat /opt/so/log/so-firewall.log
ss -tlnp                                    # confirm a port is actually listening
```

From another system (e.g. DC01, PowerShell): `Test-NetConnection -ComputerName 192.168.50.30 -Port 8220`.

5.5 Desktop launchers

Four one-click launchers (symlinked to `~/Desktop/` and `~/.local/share/applications/`), replacing several earlier, discarded attempts (2026-07-09, -10, -12).

Launcher	Runs	Purpose
Pentest Lab Start	`konsole --hold -e lab-start.sh`	Starts all 7 VMs in order
Pentest Lab Stop	`konsole --hold -e lab-stop.sh`	Stops all 7 VMs cleanly
SSH Alle Machines	`lab-ssh-all.sh`	One Konsole window, one real independent tab per machine
Homelab VM Manager	`flatpak run org.virt_manager.virt-manager --connect qemu:///system`	virt-manager on the correct (system) connection
Security Onion Operator	(separate — §5.3)	Persistent browser session for SO/Kibana/Fleet

`--hold` keeps Start/Stop windows open after completion so the summary stays readable. All terminal launchers use **Konsole** specifically — Bazzite doesn't have `gnome-terminal` installed, and earlier launchers written for it silently did nothing.

Why SSH Alle Machines isn't just `konsole -e`: the first version used `tmux` in a single tab — impractical (only one connection visible, password entry required switching tmux windows). Rebuilt on Konsole's native `--tabs-from-file` with `--separate` (without which new invocations pile tabs onto whatever Konsole window is already open). Because the script itself must call `konsole`, the launcher's `Exec=` runs the script directly rather than wrapping it in an outer `konsole -e` (doing both opens two windows). **Known cosmetic quirk:** an extra tab titled after the working directory sometimes appears — this is Konsole's own session-restore behavior, not something the script creates; harmless.

SSH config (`~/.ssh/config`, rebuilt 2026-07-12 from live-verified addresses):

Alias	Host	User	Verified
`opnsense`	192.168.50.1	root	SSH confirmed, password prompt
`dc01`	192.168.50.10	Administrator	Port confirmed open; username not fully verified
`security-onion`	192.168.50.30	socadmin	Confirmed, key auth works
`kali`	192.168.50.50	blue1	Confirmed, key auth works
`ubuntu-server`	192.168.50.40	ubuntu	Port confirmed open; username not fully verified

WIN11-01 was checked and excluded (port 22 doesn't respond — normal for a Windows 11 client). Two older, stale aliases (`onion` at .9, `kali` at .157) were removed — both addresses had moved.

A real bug found and fixed while building this: testing `lab-start.sh` / `lab-stop.sh` exposed a **libvirtd deadlock** in the existing `soc-mirror.sh` libvirt hook — it called back into `virsh` *synchronously* from inside libvirtd's own event handling, which could hang libvirtd until manually restarted. Fixed by running the reconciler fully detached via `systemd-run --no-block --collect` (falls back to `nohup`). Verified: a full 7-VM stop/start cycle now completes in under two minutes with `virsh` staying responsive throughout.

5.6 Quick reference — common operations

```
# Generate a Sysmon test event on DC01 (via SSH) to confirm telemetry flow
Start-Process -FilePath 'cmd.exe' -ArgumentList '/c echo test' -Wait
# Then check in Security Onion Hunt (window: Last 15 Minutes):
host.name:"dc01" AND event.dataset:"windows.sysmon_operational"

# Traffic-mirroring status
~/Homelab/scripts/soc-mirror.sh --status

# Security Onion firewall log
ssh security-onion "cat /opt/so/log/so-firewall.log | tail -20"
```

6. Detection use cases & incident response

6.1 What this SOC should detect

Status labels: ✅ present (Security Onion has built-in Suricata/Sigma coverage, works once traffic/logs arrive) · ⚠️ should be present via default rulesets but not yet deliberately triggered/confirmed in this lab · ❌ requires custom configuration/rules not yet built.

Network reconnaissance (Suricata/Zeek):

Detection	Status
ICMP ping sweeps	⚠️
TCP SYN/FIN/NULL/XMAS scans	⚠️ (requires traffic to actually reach Security Onion via `monitor-net`)
UDP scans	⚠️
OS fingerprinting / banner grabbing	⚠️ (visible in Zeek connection logs, no dedicated alert rule)

Login attempts (Windows Event Log / Sysmon / Suricata):

Detection	Status
SSH brute force	⚠️ (Suricata rules present)
FTP brute force	⚠️
SMB brute force	⚠️ (relevant especially toward DC01; Windows Security-channel logs contribute input since 2026-07-13)
Suspicious DNS requests	⚠️ (Zeek logs network DNS; Sysmon event ID 22 on DC01 logs local DNS queries since 2026-07-13)

Known attack techniques:

Detection	Status
Known exploit signatures	⚠️ (Suricata ET Open / other rulesets, depending on subscription)
Reverse shells	⚠️ (network-side via Suricata, host-side via Sysmon process-creation events since 2026-07-13)
Metasploit/Meterpreter indicators	⚠️ (known Suricata signatures)
SQLi / command injection / directory traversal	⚠️ (Suricata web-attack rulesets)

Host-based detection (Sysmon/Elastic Defend on DC01, since the 2026-07-13 install):

Sysmon Event ID	Meaning	Status
1	Process Create	✅ confirmed with test events
3	Network Connection	✅ confirmed with test events
11	File Create	✅ confirmed with test events
22	DNS Query	✅ confirmed with test events
Other (registry, image load, etc.)	Various	⚠️ covered by the SwiftOnSecurity config, not individually tested

Elastic Defend (Elastic Agent's built-in endpoint security) also produces its own detections on DC01 since the 2026-07-13 firewall fix.

How to test a detection (Purple Team style): run a harmless version of the attack from Kali or another lab VM, then check Security Onion's Hunt/Detections page. Example — simulated SSH brute force against ubuntu-server-01:

```
for i in 1 2 3 4 5; do
  ssh -o ConnectTimeout=3 -o BatchMode=yes nonexistentuser@192.168.50.40
done
```

Then in Hunt: `destination.ip:"192.168.50.40" AND event.dataset:*ssh*`

6.2 Incident response runbook

A fixed, ordered procedure for any Security Onion alert — real attack, test, or Purple Team exercise. Steps are not to be skipped, even for something that looks harmless.

Step 1 — Confirm something real happened. Go to Security Onion → Alerts/Detections. Which rule fired? Source/destination IP? When (note: SO displays local Dutch time, +02:00 in summer)? How often?

Step 2 — Put it in context. In Hunt, search broadly around the same IP/time:

```
source.ip:"<IP>" OR destination.ip:"<IP>"
```

Is this IP a system that's supposed to exist in this lab (§9)? Was a deliberate test/exercise running at that time (check `docs/daily/`)? Did anything else happen around the same time?

Step 3 — Look at the host itself (if it's a known lab system with SSH access): `ssh dc01` and review local logs/processes, and (for DC01 specifically) Sysmon events around the timestamp. **Boundary:** investigating (reading, searching, comparing) is always fine to do independently. Changing configuration, restarting services, or isolating something requires — unless already agreed otherwise — a short explanation of intent first.

Step 4 — Record what you found. Even if it turns out to be a test or false alarm, write it up in that day's report (`docs/daily/YYYY-MM-DD/rapport.md`): what the alert was, what was checked, the conclusion (real/test/false alarm/unclear). If it's a genuine *new* technical problem (not an attack, e.g. the DC01 Fleet

case), also create a `docs/troubleshooting/` document with the same evidence standard as §7.6: problem, cause, fix, how it was tested.

Step 5 — If it's a real threat (even inside this isolated lab): 1) don't panic — the lab is isolated from the rest of the network; 2) consider isolating the system (e.g. detach its network interface via `virsh`) — explain first unless broader approval already exists for that specific system/action; 3) preserve evidence before cleaning up (screenshots, event IDs, IPs, times); 4) document as in Step 4.

Quick Hunt searches:

Looking for	Query
Everything from one host	`host.name:"<name, lowercase>"`
Only Windows events from a host	`host.name:"<name>" AND event.module:"windows"`
Sysmon events specifically	`host.name:"<name>" AND event.dataset:"windows.sysmon_operational"`
Traffic to/from an IP	`source.ip:"<ip>" OR destination.ip:"<ip>"`
Last 15 minutes	set the time picker (top right) to "Last 15 Minutes"

⚠️ **Case note:** hostnames are indexed lowercase in Security Onion (`dc01`, not `DC01`), even though the Fleet page displays it capitalized.

---

7. Troubleshooting history

Real problems encountered building this lab, in chronological/topical order, with evidence and fixes. This section preserves technical detail exactly — do not treat the summaries below as a substitute for the source documents when replaying a fix.

7.1 QEMU guest agent not configured

System: DC01. **Symptom:** `sudo virsh qemu-agent-command DC01 '{"execute":"guest-ping"}'` → `error: argument unsupported: QEMU guest agent is not configured`. **Cause:** the guest agent wasn't installed/enabled inside the VM, and the QEMU channel wasn't configured. **Fix:** install the guest agent inside the guest OS, enable its service, configure the VM channel, restart, retest. **Lesson:** a running VM doesn't imply guest integration works — test it explicitly before relying on guest-management features.

7.2 virt-manager VM setup issues

System: Bazzite host. Storage: standardized on qcow2 (snapshots, thin provisioning, flexible management). One image converted from VMDK via `qemu-img convert -f vmdk -O qcow2`. Display: SPICE fullscreen/resolution issues traced to guest-integration tooling rather than hardware. **Lessons:** document VM hardware settings, snapshot before risky changes, test networking before installing services on a new VM.

7.3 OPNsense network debugging

Systems: OPNsense, DC01, Security Onion. General findings: a firewall problem can *look* like an application, server, or DNS problem — always verify network layers (interface → IP → gateway → DNS → firewall rules) from the bottom up before assuming a higher-layer cause. DNS misconfiguration was repeatedly the real root cause behind apparent Active Directory problems. **Test commands:** `ping 192.168.50.1` (OPNsense), `ping 192.168.50.10` (DC01), `nslookup <hostname>`.

7.4 Security Onion installation

System: Security Onion VM. Resourcing note: SOC platforms need materially more RAM/CPU/storage than an ordinary server — under-provisioning caused early performance concerns. Admin account: `socadmin`, kept separate from personal/daily accounts. Core lesson: a SOC platform is only as useful as the telemetry reaching it — network visibility and log-source planning have to happen *before* detection engineering, not after.

7.5 DC01 Active Directory setup

System: DC01, 192.168.50.10. Confirmed early: DNS is the foundation Active Directory depends on — most AD problems trace back to DNS. Setup order: static IP → hostname → network connectivity verified → AD DS + DNS roles installed → promoted to Domain Controller (new domain, `pentest.lab`)

→ account tiers planned (Domain Admin / Server Admin / SOC Admin / Standard User). **Lesson:** take a snapshot before promoting to DC — this step is hard to cleanly undo.

7.6 DC01 Fleet health & Sysmon telemetry (2026-07-13)

The largest, most thoroughly evidenced troubleshooting case in the project — three independent, stacked root causes behind one symptom ("DC01 Offline in Fleet, no Windows/Sysmon telemetry").

This was a recurring issue. On 2026-07-10, the same symptom (DC01 unable to reach Fleet Server on 8220; Elastic Defend stuck `DEGRADED`) was already diagnosed once and "fixed" with a direct, non-persistent rule:

```
sudo iptables -I DOCKER-USER 1 -s 192.168.50.10/32 -p tcp --dport 8220 -j ACCEPT
```

This bypassed Security Onion's salt-managed firewall config entirely, so any later `so-firewall apply` or SO reboot silently discarded it — which is almost certainly why the identical symptom reappeared by 2026-07-13. Elastic Defend's `DEGRADED` status was investigated on 07-10 without finding a network cause and was deliberately parked; it turned out to share the same root-cause category (a missing hostgroup, `endgame` /port 3765).

Root cause 1 — firewall hostgroups. DC01 could reach TCP 443 (web/Kibana, gated by `analyst`) but not 8220 (Fleet checkin) or 5055 (data ingest) — connections timed out. `ss -tlnp` on Security Onion confirmed the ports *were* listening, ruling out "service down" and pointing at a firewall drop. `/opt/so/log/so-firewall.log` showed DC01 had only ever been added to `analyst` (on 2026-07-10), never to `elastic_agent_endpoint`, `beats_endpoint`, or `endgame`. **Fix:**

```
so-firewall includehost elastic_agent_endpoint 192.168.50.10
so-firewall includehost beats_endpoint 192.168.50.10
so-firewall includehost endgame 192.168.50.10
so-firewall apply
```

Additive only — DC01 was not removed from `analyst`. **Verified:** `Test-NetConnection` to 8220/5055/3765 all `True`; Fleet's `endpoint` component went from `DEGRADED` (`Unable to connect to output server`) to `HEALTHY`; survived a full Security Onion reboot (rules re-applied from the salt-managed pillar, not a runtime-only iptables state) and a full DC01 reboot.

Root cause 2 — ~9-hour clock skew on DC01. After the firewall fix, DC01 checked into Fleet correctly — but after a DC01 reboot, all Fleet components got stuck in `STARTING` indefinitely. Comparison: `DC01 UTC: 2026-07-13 10:55:19` vs `Security Onion UTC: 2026-07-13 01:55:20`. `w32tm /query /status` showed `Source: Local CMOS Clock` — no NTP sync configured; as the domain's PDC Emulator, DC01 is expected to be an authoritative time source rather than an ordinary NTP client, so it was never pulling correct time from anywhere. First attempt (`w32tm /config /manualpeerlist:"pool.ntp.org,0x8" ...` + forced resync) fixed the current boot but **the skew came back on a second reboot** — the registry config persisted, but the service never synced early enough after boot. Real cause: `vmictimesync` (a Hyper-V-compatible time-sync integration service, also active under QEMU/KVM's Hyper-V-enlightenment emulation) was set to `Manual` start and overriding/racing NTP at boot. **Final, persistent fix:**

```
Stop-Service vmictimesync -Force
Set-Service vmictimesync -StartupType Disabled
w32tm /resync /force
```

Plus a defense-in-depth scheduled task forcing `w32tm /resync /force` 30 seconds after every startup. **Verified:** two additional full reboots with zero manual intervention, clock matched Security Onion's within 1 second each time. **Side effect discovered:** the clock skew had also been silently poisoning the Logstash → Elasticsearch pipeline — events shipped while the clock was wrong got `@timestamp` values ~9 hours in the future, which Elasticsearch rejected, routing them to Logstash's dead-letter queue (`dlq_routed: 1917`, confirmed static/non-growing after the clock fix — old poisoned backlog, not ongoing). This is the real reason winlog-sourced datasets never appeared in Security Onion even though the Elastic Agent itself reported those inputs `HEALTHY` — the agent successfully shipped the events, but Elasticsearch silently rejected them downstream. **Lesson:** an agent-side `HEALTHY` status only proves the agent believes it's shipping data, not that it was accepted.

Root cause 3 — Sysmon never installed. `Get-Service *sysmon*` → nothing; the Sysmon-Operational event log didn't exist. Security Onion's own Fleet policy already explicitly subscribed to the `windows.sysmon_operational` dataset — it was waiting for data that could never arrive. **Fix:** downloaded Sysmon from the official Sysinternals source (signature verified: `Valid`, `CN=Microsoft Windows Publisher`), downloaded the SwiftOnSecurity config (schema 4.50, still the de-facto SOC standard — Sysmon's schema is additive, so this remains valid on newer builds), installed via `Sysmon64.exe -accepteula -i sysmonconfig.xml` (installed v15.21, schema 4.91). **Verified:** local test events (process create, DNS query, file create, network connection) all logged with correct event IDs (1/22/11/3); Fleet's `windows.sysmon_operational` sub-stream went `STARTING` → `HEALTHY` without an agent restart; confirmed end-to-end in Hunt matching the exact test actions; survived an Elastic Agent service restart (~5 min to stabilize vs ~2 min for a VM reboot) and two full DC01 reboots (1,816 new sysmon events in the 15 minutes after the second reboot, including the literal boot process tree).

Additional change: DC01's Windows timezone set to `W. Europe Standard Time` per request — cosmetic only, the underlying UTC clock remains authoritative.

Full verification summary:

Check	Result
Fleet UI shows DC01	**Healthy**
All Fleet components (endpoint, winlog, osquery, windows/metrics, filestream)	HEALTHY
TCP 8220/5055/3765 reachable from DC01	Confirmed
Firewall config persistent across SO reboot	Confirmed
DC01 clock matches SO clock after 2 independent reboots, no manual fix	Confirmed
Sysmon service + event channel	Running / exists
Sysmon events reach Security Onion	Confirmed via Hunt
Survives Elastic Agent restart	Confirmed
Survives DC01 reboot (×2)	Confirmed
Survives Security Onion reboot	Confirmed

Rollback procedures (kept for reference, not expected to be needed):

```
# Firewall — removes DC01 from ALL hostgroups, then optionally re-add only analyst:
so-firewall removehost 192.168.50.10
so-firewall includehost analyst 192.168.50.10
so-firewall apply
```

```
# Clock / vmictimesync
Set-Service vmictimesync -StartupType Manual
Unregister-ScheduledTask -TaskName 'Force-NTP-Resync-At-Boot' -Confirm:$false
w32tm /config /syncfromflags:domhier /update
Restart-Service w32time
```

```
# Sysmon
C:\Tools\Sysmon\Sysmon64.exe -u force
Remove-Item -Recurse -Force C:\Tools\Sysmon
```

```
# Timezone
Set-TimeZone -Id 'Pacific Standard Time'   # or whatever the original was
```

General lessons from this case:

- A "Healthy/Offline" Fleet status can have multiple independent, stacked root causes — each fix can look complete until the next reboot re-exposes the next layer. Validate across a real reboot cycle, not just current running state.
- Agent-reported `HEALTHY` proves the agent believes it's shipping data, not that Elasticsearch accepted it — the Logstash DLQ counter was the key signal here.
- Domain Controllers (especially the PDC Emulator) don't behave like ordinary NTP clients by default.

- Security Onion's per-hostgroup firewall model means reachability on one port says nothing about reachability on a different, differently-gated port — each integration's hostgroup requirement must be checked explicitly.

7.7 The `soc-mirror.sh` libvirtd deadlock (found while building the launchers)

See §5.5 — a synchronous callback from a libvirt qemu hook into `virsh` could deadlock libvirtd itself. Fixed by detaching the reconciler via `systemd-run --no-block --collect`.

7.8 libvirt/OPNsense DHCP conflict (2026-07-09)

See §8 — libvirt's own DHCP on `pentest-lab` collided with OPNsense's, and the `virbr10` bridge itself claimed OPNsense's own IP (192.168.50.1). Fixed by stripping libvirt's `<ip>` / `<dhcp>` sections from the network definition entirely (OPNsense became the sole DHCP/DNS/gateway layer) and giving the Bazzite host a separate management IP (192.168.50.254/24) on `virbr10`, restored automatically at boot by `fix-virbr10-ip.service`.

---

8. Project timeline

Period	Project name	What happened
Before 2026-07-05	Fortress Bazzite	Initial setup: virt-manager, Kali/Debian, Ubuntu Server, Metasploitable2, Docker, Juice Shop, Burp Suite, Metasploit. Design document written planning a Suricata + Zeek + Wazuh stack.
2026-07-03 – 07-07	Fortress Bazzite	Kali/Metasploitable2 VMs stood up (VirtIO incompatibility fixed via i440FX+SATA+e1000); first `nmap` scans; isolated `pentest-lab` network built (virbr10, 192.168.50.0/24). Metasploitable2 enumerated (Nmap, Gobuster, Nikto, smbclient, enum4linux-ng). Metasploit set up; `vsftpd_234_backdoor` exploited → Meterpreter session. Ubuntu Server + Docker + Juice Shop added; first vuln found via Burp Suite (SQLite disclosure). Intel AX210 wifi passed through to Kali via PCI-passthrough (requests for monitor-mode/injection/cracking were declined by the assistant; only theoretical/defensive guidance given). BIOS VT-x/VT-d confirmed enabled (07-05). Suricata installed directly on the Bazzite host itself (not a VM), watching `enp6s0` and `virbr10`; custom test rules (ICMP, Nmap SYN scan) confirmed via `eve.json` / `fast.log`; a bash/Python watcher script built, with plans for a full curses/Qt dashboard ("FORTRESS IDS v2") — this became a separate side project (`/var/home/Joost/FORTRESS/`, still present on disk, not part of this repo). OPNsense installed for the first time (07-07); first attempt hung in live-media mode, redone with ZFS; WAN connectivity/DHCP issues left open at period end.
2026-07-08	Transition to "SOC Homelab"	⚠️ Reconstructed from limited evidence (one firewall-log line). The full 192.168.50.0/24 network was added to Security Onion's `analyst` hostgroup.
2026-07-09	—	First git commit (`3ab5374`, 18:53:51, "Initial SOC homelab baseline"). Major network-recovery day: found and fixed a libvirt-vs-OPNsense DHCP conflict and a `virbr10` IP collision with OPNsense itself (see §7.8); Kea DHCP reservations created for all VMs; desktop launchers fixed for KDE/Konsole (were written for GNOME Terminal); lab-start/stop scripts corrected to real VM names including the `ATTACK-Kali` space. End of day: ping/SSH to OPNsense working, Start/Stop launchers working, clean DNS/DHCP structure (OPNsense as sole authority).
2026-07-10	—	DC01 fully built (Windows Server + AD DS + DNS, promoted to DC, users/accounts created) and connected to Security Onion via Fleet (agent enrolled 11:18:17). First Fleet connectivity problem found and "fixed" with a non-persistent `iptables` rule (see §7.6 for why this didn't last). Elastic Defend `DEGRADED` investigated, no cause found, deliberately parked.
2026-07-11	—	Full documentation structure created: README, project rules, AI access policy, network/server/AD documentation, troubleshooting history.
2026-07-12	—	Secret (`Secure/SOC-Secure.img`) scrubbed from git history. Desktop launchers built properly (replacing several earlier discarded attempts). Event-driven traffic mirroring (`soc-mirror.service`) rewritten, fixing the libvirtd deadlock bug. Browser automation for Security Onion built.
2026-07-13	—	DC01's Fleet outage fully, persistently resolved (firewall hostgroups + clock/`vmictimesync` + Sysmon install — §7.6). Documentation structure substantially expanded: daily reports, asset inventory, glossary, network/port reference, incident-response runbook, detection use cases, and this master document. Commit `2f40b20`

Period	Project name	What happened
		("Fix DC01 Fleet health and add Sysmon telemetry") made but **not pushed**, per the user's standing instruction to never push without explicit permission.

---

9. Asset inventory

Full detail: `docs/ASSET_INVENTORY.md`*. Reliability labels: ✅ verified this session / hard evidence, ⚠️ not reverified this session.*

Physical host

Property	Value	Status
OS	Bazzite Linux	✅
CPU	Intel Core i9-11900K	⚠️ from the 2026-07-05 Fortress Bazzite document
GPU	NVIDIA RTX 3090	⚠️ same source
WiFi	Intel AX210 (Wi-Fi 6E), PCI-passthrough to Kali	⚠️ same source
Virtualization	KVM, QEMU, libvirt, virt-manager	✅
Other	Docker (OWASP Juice Shop etc.)	⚠️ same source

Virtual machines

VM name (virsh)	IP	SSH alias	SSH user	OS / role	Status
`OPNsense-FW`	192.168.50.1	`opnsense`	`root`	OPNsense firewall/gateway	✅
`DC01`	192.168.50.10	`dc01`	`Administrator`	Windows Server 2022, AD DC (PDC Emulator), domain `pentest.lab`	✅
`WIN11-01`	192.168.50.20	*(none)*	—	Windows 11 workstation	✅ (IP), ⚠️ (role)
`SOC-SecurityOnion`	192.168.50.30	`security-onion`	`socadmin`	Security Onion 3.1.0 standalone	✅
`ubuntu-server-01`	192.168.50.40	`ubuntu-server`	`ubuntu`	Ubuntu Server	✅ (IP), ⚠️ (role)
`ATTACK-Kali` (leading space)	192.168.50.50	`kali`	`blue1`	Kali Linux, Red Team workstation	✅
`Target-Metasploitable2`	⚠️ unverified	*(none)*	—	Metasploitable2, vulnerable target	⚠️

Virtual networks

Name	Type	Purpose	Status
`pentest-lab`	Isolated bridge	Main network, 192.168.50.0/24	✅
`monitor-net`	Isolated bridge, no IP addressing	Traffic mirroring to Security Onion	✅
`default`	Standard libvirt network	Not used by lab VMs	✅

Software versions

Component	Version	Status
Security Onion	3.1.0	✅
Elastic Agent (on DC01)	9.3.3+build202604082258	✅
Sysmon (on DC01)	15.21 (schema 4.91)	✅, installed 2026-07-13
Sysmon config	SwiftOnSecurity, schema 4.50	✅
Windows Server (DC01)	Windows Server 2022 Standard Evaluation	✅

Desktop launchers (on the Bazzite host)

Launcher	Purpose
Pentest Lab Start	Starts all lab VMs in order
Pentest Lab Stop	Stops all lab VMs cleanly
SSH Alle Machines	Konsole tab per system with SSH access
Homelab VM Manager	virt-manager on `qemu:///system`
Security Onion Operator	Browser window with all key SO/Kibana/Fleet pages as tabs

Access methods (no passwords stored here)

Method	Used for
SSH keys (`~/.ssh/config`)	opnsense, dc01, security-onion, kali, ubuntu-server
Browser session (Playwright, dedicated profile)	Security Onion web UI, Kibana, Fleet
`virsh -c qemu:///system`	VM management from the hypervisor itself

Open verification items

- Exact IP of `Target-Metasploitable2` .
- Precise role/configuration of `WIN11-01` and `ubuntu-server-01` .

- Current hardware specs of the Bazzite host (CPU/GPU/WiFi from the 2026-07-05 document, not rechecked).

---

10. Glossary

Active Directory (AD) — Microsoft's system for managing users, computers, and permissions within an organization ("domain"). Runs on **DC01** here; domain is `pentest.lab`.

Agent (see also Elastic Agent) — a small program running on a computer that collects and forwards data (logs, metrics, events) to a central system.

Beats — Elastic's family of lightweight data shippers (e.g. Winlogbeat, Metricbeat). In modern Elastic deployments these typically run *inside* Elastic Agent rather than standalone.

CEST / CET — Central European (Summer) Time. CET = UTC+1 (winter), CEST = UTC+2 (summer). The Netherlands uses CEST late March–late October.

Dead Letter Queue (DLQ) — where Logstash puts documents it couldn't store in Elasticsearch (e.g. bad timestamp). A growing DLQ counter signals a problem downstream even if the sender sees no error.

Domain Controller (DC) — a server running Active Directory that manages users/computers in a domain. Here: **DC01**.

Elastic Agent — the program running on DC01 (and other systems) that collects Windows Event Logs, Sysmon data, metrics, etc. and ships them to Security Onion. Composed of multiple components, each with its own health status.

Elastic Defend (a.k.a. "Endpoint") — Elastic Agent's endpoint-security component (EDR-like). Uses its own dedicated Fleet connection (port 3765, hostgroup `endgame`), separate from ordinary log ingest.

Elasticsearch — the database Security Onion stores all logs/events in. Internally always UTC-timestamped regardless of the server's OS timezone setting.

Endpoint — usually: a computer monitored by Elastic Agent (e.g. DC01). Can also mean "Elastic Defend" specifically — context matters.

Firewall hostgroup / portgroup — see §5.4. Hostgroup = named group of IPs; portgroup = named group of ports; Security Onion's firewall links the two to decide who can reach what.

Fleet — the Kibana/Elastic component for centrally managing and monitoring all Elastic Agents. Agent states: `online`, `degraded`, `offline`, `unhealthy`.

Fleet Server — the server side of Fleet, where agents check in. Runs on Security Onion, port 8220.

Hunt — Security Onion's own log/event search interface (similar to Kibana Discover, with SO's own UI wrapped around it). Left-hand menu in the SO web interface.

KVM / QEMU / libvirt — the virtualization stack running every VM in this lab. KVM = kernel technology, QEMU = hardware emulation, libvirt = the management layer (e.g. the `virsh` command).

NTP (Network Time Protocol) — protocol for syncing a computer's clock to a trusted network time source. DC01 uses `pool.ntp.org`.

PDC Emulator — an Active Directory role: the "Primary Domain Controller Emulator" is normally the authoritative time source for the whole domain. DC01 holds this role (the lab's only DC), which explains why it behaves differently from an ordinary NTP client.

Security Onion — the platform used as this lab's SOC. Bundles Suricata, Zeek, Elasticsearch, Kibana, and Fleet. Runs on VM `SOC-SecurityOnion` (192.168.50.30).

SIEM — Security Information and Event Management: collects, combines, and makes logs searchable across sources to find security incidents. Security Onion is this lab's SIEM.

Sigma rule — a general-purpose log detection rule format (analogous to Suricata rules for network traffic), describing what pattern is suspicious.

so-firewall — the CLI tool for managing Security Onion's own firewall (adding/removing hostgroup membership, applying changes). See §5.4.

Sysmon (System Monitor) — a free Microsoft Sysinternals tool logging detailed Windows events (process starts, network connections, file creation, DNS queries, and more) far beyond the standard Windows Event Log. Installed on DC01 on 2026-07-13 with the SwiftOnSecurity config.

UTC (Coordinated Universal Time) — the global time standard with no timezone offset. Elasticsearch always stores timestamps in UTC internally. Dutch time (CEST in summer) is UTC+2.

vmictimesync — a Windows service that syncs the system clock with the hypervisor (originally for Hyper-V, but also active under QEMU/KVM when it exposes similar synthetic-time devices). On DC01, this service reset the clock incorrectly at every reboot even after NTP was correctly configured — disabled 2026-07-13.

Zeek (formerly Bro) — a network analysis tool that records *what* happens on the network (DNS traffic, HTTP requests, TLS certificates, connections), as opposed to Suricata which focuses on detecting known attack patterns. Part of Security Onion.

---

11. Current project status / what's next

As of 2026-07-13. Full detail and how to keep this current: `docs/PROJECT_STATUS.md`.

✅ Done (working, tested, documented)

- Base infrastructure: OPNsense, DC01, Security Onion, Kali, WIN11-01, ubuntu-server-01, Metasploitable2 — all running on `pentest-lab` (192.168.50.0/24).
- Event-driven traffic mirroring to Security Onion (no polling/timer), survives VM restarts.
- Active Directory operational (DC01, domain `pentest.lab`).
- Security Onion operational: web UI, Kibana, Fleet, Hunt.
- DC01 **Healthy** in Fleet with working Windows Event Log, Sysmon, and Elastic Defend telemetry — confirmed to survive an Elastic Agent restart, two DC01 reboots, and a Security Onion reboot.
- Passwordless SSH to security-onion and kali (opnsense/dc01/ubuntu-server still password-prompt).
- Four working desktop launchers (start/stop lab, SSH to all machines, VM manager, Security Onion browser operator).
- Read-only web-audit script (`scripts/soc-web-audit.sh`) reporting Fleet status, data streams, and Grid status.
- The documentation structure this file is compiled from.

⚠️ Not yet done / open

- Exact IP of `Target-Metasploitable2` not verified.
- Precise role/configuration of `WIN11-01` and `ubuntu-server-01` not documented in detail.
- Security Onion's own OS-level timezone still UTC (cosmetic only — the web UI already shows Dutch time); requires broader root access than the current narrow `so-firewall` sudo scope.
- DHCP ranges and DNS forwarders not documented in detail.
- OPNsense and ubuntu-server may still lack passwordless SSH keys (not rechecked this session).

❌ Planned (from the original Fortress Bazzite design, largely still relevant)

- **Detection engineering:** most network/host detections are ⚠️ "should work, not yet deliberately confirmed" — see §6.1.
- **Dashboard:** the original plan wanted host health (CPU/GPU/RAM/temps), virtualization status, security metrics (open alerts, top source/destination IPs, portscan detections), and even gaming metrics (latency, packet loss) in one view. Security Onion's own dashboards cover the security half; a combined host/gaming dashboard is not built.
- **Alert levels:** four planned tiers (INFO/WARNING/HIGH/CRITICAL) with automatic HIGH/CRITICAL forwarding to Discord/Telegram — not implemented.
- **Other:** VLAN segmentation, a separate management network and a separate attack network, additional Windows clients, honeypots, periodic Purple Team validation exercises (attack yourself, confirm Security Onion sees it).

Keeping this document (and the project) current

On every important change: 1) update the relevant specific document (troubleshooting/guide/network/server doc), 2) update `CHANGELOG.md`, 3) update `docs/PROJECT_STATUS.md`, 4) write a daily report in `docs/daily/YYYY-MM-DD/`, 5) **update this master document** so it doesn't drift from the sources it was compiled from.

Document index (sources compiled into this file)

Root: `README.md`, `LAB_OVERVIEW.md`, `PROJECT_RULES.md`, `AI_ACCESS_POLICY.md`, `NETWORK.md`, `SERVERS.md`, `SECURITY.md`, `ACTIVE_DIRECTORY.md`, `CHANGELOG.md`. `docs/`: `INDEX.md`, `ASSET_INVENTORY.md`, `GLOSSARY.md`, `PROJECT_STATUS.md`, `decisions/*.md`, `guides/*.md` (11 files), `troubleshooting/01` – `06`, `chat_history/*.md` (6 files), `daily/*/` (2026-07-03 through 2026-07-13).

Note: `pentest.lab - by Joost Hebly.md` (repo root) is an earlier, unmerged concatenation of the same source files, generated for the portfolio deliverable — this document supersedes it as the synthesized reference; that file remains useful as the literal, unedited per-file archive.